Categorial Grammar and String Diagrams

Categorial Grammar, Pregroup Types, and String Diagrams

From Phrase Structure Rules to Algebraic Type Assignment

Traditional generative grammar assigns sentence structure through phrase-structure rules that recursively combine constituents into larger units. Categorial grammar inverts this perspective: rather than specifying construction rules, it assigns each lexical item a *type* that encodes its combinatory potential. A transitive verb like "chases" is not described as "a word that takes a subject NP and an object NP to form a VP" but rather as having the type $(n\backslash s)/n$—an entity that seeks a noun to its right (the object) and a noun to its left (the subject) in order to produce a sentence.

Lambek's Residuated Syntactic Calculus

The algebraic foundations were laid by Lambek [-@lambek1958mathematics], who introduced a *residuated* structure on syntactic types: two binary operations, the left residual $\backslash$ and the right residual $/$ of the multiplicative connective $\otimes$ (concatenation). The key axiom is the residuation law:

$$A \otimes B \leq C \quad \Longleftrightarrow \quad A \leq C/B \quad \Longleftrightarrow \quad B \leq A\backslash C$$

This law captures the fundamental duality of syntax: a verb of type $(n\backslash s)/n$ *consumes* noun arguments to *produce* a sentence. The resulting **Lambek calculus** is a non-commutative intuitionistic linear logic—non-commutative because word order matters, and linear because each resource (word) is used exactly once.

Pregroup Grammars and Compact Closed Structure

Lambek [-@lambek2004fine] later simplified the framework to **pregroup grammars**, where each type a has a left adjoint a^l and a right adjoint a^r satisfying:

String Diagrams for Syntactic Derivation

The pivotal connection between categorial grammar and visualization comes from Joyal and Street [-@joyalstreet1991geometry], who proved that morphisms in monoidal categories can be faithfully represented as *string diagrams*—planar graphs where:

- **Wires** represent types (objects of the category)
- **Boxes** represent words or operations (morphisms)
- **Vertical composition** represents sequential application
- **Horizontal juxtaposition** represents the tensor product (concatenation)
- **Cups and caps** represent the contraction/expansion of a pregroup

1 shows a DisCoCat-style string diagram for the sentence "Alice chases Bob," generated by our visualization pipeline. The noun wires connect to the verb's input slots, and the sentence wire emerges from the verb's output—the entire derivation from word types to sentence type is visible at a glance.

Case Marking in Type-Logical Categorial Grammar

Case-marked noun phrases receive compound types that encode both their grammatical role and their combinatory potential. In a nominative–accusative language, we can assign:

Expression	Type	Gloss
"Alice" (subject)	n_{NOM}	Noun, nominative-marked
"the ball" (object)	n_{ACC}	Noun, accusative-marked
"kicks"	$(n_{\text{NOM}} \backslash s)/n_{\text{ACC}}$	Seeks ACC right, NOM left
"to Bob" (recipient)	$(s \backslash s)/n_{\text{DAT}}$	Modifies sentence via DAT argument

The subscripts refine the basic noun type n with case features, ensuring that "kicks" selects a nominative subject and an accusative object. Mismatch in case features blocks the derivation, modeling grammaticality

The Curry–Howard Correspondence and Proof-Theoretic Semantics

A deep structural parallel underlies the Lambek calculus: derivations of syntactic types correspond to proofs in intuitionistic logic (via the Curry–Howard isomorphism), and therefore to programs in a typed lambda calculus. Specifically:

Syntactic Domain	Logical Domain	Computational Domain
Types $(A/B, A\backslash B)$	Propositions	Types
Derivations	Proofs	Programs (λ-terms)
Cut elimination	Proof normalization	β-reduction
Commutativity of cuts	Confluence of rewriting	Church–Rosser property

This correspondence has two important consequences for our framework:

1. **Semantic compositionality follows from syntactic**

Monadic Semantics for Sublexical Root Syntax

Song [-@song2022act; -@song2022blog] extends the categorial framework with a **monadic semantics** for root syntax, modeling the sublexical decomposition of verb roots using monads in a syntactic category. This approach captures the intuition that a verb like "break" has internal compositional structure—a causative layer and a result-state layer—that interacts with case assignment. The monad encapsulates the complexity of root meaning within a single categorical construction, allowing the type-logical grammar to handle both lexical and syntactic composition uniformly.

The monadic approach connects naturally to graded type theory: Asudeh and Giorgolo [-@asudeh2020graded] develop a monadic semantics for evidentiality that uses graded effects to track the epistemic status of propositions—a pattern that extends to case, where the "grade" of a morphism could encode the strength of a semantic role assignment.

Passivization as Algebraic Type Permutation

One of the most revealing case-theoretic operations is **passivization**, which transforms an active transitive clause into a passive construction by promoting the patient to subject position and (optionally) demoting the agent to an oblique role. In our categorical framework, passivization is not an ad hoc syntactic transformation but a precise *type permutation*—a swap of the noun types feeding into the verb's pregroup derivation.

In a pregroup grammar, the active transitive "Alice chases Bob" has the type reduction:

$$n_{\mathsf{NOM}} \cdot (n^r \cdot s \cdot n^l) \cdot n_{\mathsf{ACC}} \to s$$

Passivization permutes the noun arguments, yielding "Bob is chased by Alice" with the swapped type assignment:

$$n_{\mathsf{ACC}} \cdot (n^r \cdot s \cdot n^l) \cdot n_{\mathsf{NOM}} \to s$$